Socle Filtration via Littlewood–Richardson Coefficients

Pablo Santos Guerrero

November 15, 2025

Abstract

We give a self- contained, rigorous exposition of a Python program that computes the $(k{+}1)^{\text{st}}$ socle layer of a tensor product of induced modules. The algorithm relies on combinatorial representation theory: partitions, skew shapes, Littlewood–Richardson (LR) coefficients, and Yamanouchi (lattice) words. All mathematical notions are introduced together with the corresponding code fragments; the overall flow of the program is described in detail.

Contents

1 Mathematical background

1.1 Partitions and degree

A *partition* of a non-negative integer n is a weakly decreasing sequence of positive integers

$$\lambda = (\lambda_1, \lambda_2, \ldots, \lambda_\ell), \qquad \lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_\ell > 0,$$

with $|\lambda| := \sum_{i=1}^{\ell} \lambda_i = n$. In the code this is implemented by

```
def degree(partition):
    return sum(partition)
```

Listing 1: Degree of a partition

1.2 Skew shapes

If $\mu \subseteq \lambda$ (i.e. $\mu_i \leq \lambda_i$ for every row), the *skew shape* λ/μ is obtained by deleting the Young diagram of μ from that of λ. Its row lengths are

$$(\lambda_1 - \mu_1,\ \lambda_2 - \mu_2, \ldots).$$

The routine

```
def subtract_partitions(lam, mu):
    # returns None if mu is not contained in lam
    if len(mu) > len(lam) or any(mu[i] > lam[i] for i in range(
       len(mu))):
        return None
    return [lam[i] - (mu[i] if i < len(mu) else 0) for i in range
       (len(lam))]
```

Listing 2: Skew shape as a list of row lengths

produces precisely this list.

1.3 Littlewood–Richardson coefficients

Given partitions μ, ν, λ, the Littlewood–Richardson coefficient $c^{\lambda}_{\mu,\nu} \in \mathbb{Z}_{\geq 0}$ counts the number of semistandard Young tableaux of shape λ/μ having content ν and whose reading word is a *Yamanouchi (lattice)* word. Two variants are required:

- **2-order LR coefficient** $c^{\lambda}_{\mu,\nu}$ - denoted in the code by `lrcoefP`.
- **4-order LR coefficient**

$$c^{\lambda}_{\mu_1,\mu_2,\mu_3,\mu_4} = \sum_{a \vdash d_{12}} \sum_{b \vdash d_{34}} c^{a}_{\mu_1,\mu_2}\, c^{b}_{\mu_3,\mu_4}\, c^{\lambda}_{a,b},$$

where $d_{12} = |\mu_1| + |\mu_2|$ and $d_{34} = |\mu_3| + |\mu_4|$. This is implemented by `lrcoef4`.

1.4 Yamanouchi (lattice) words

A word $w = w_1 w_2 \dots w_m$ over $\mathbb{N}_{>0}$ is Yamanouchi iff for every prefix $w_1 \dots w_k$ and each integer $i \geq 1$,

$$\#\{j \leq k \mid w_j = i\} \;\geq\; \#\{j \leq k \mid w_j = i+1\}.$$

The test used in the program is

```
def is_yamanouchi(word):
    counts = {}
    for x in word:
        counts[x] = counts.get(x, 0) + 1
        for y in range(1, x):
            if counts.get(y, 0) < counts[x]:
                return False
    return True
```

Listing 3: Yamanouchi test

---

2 Algorithmic components

2.1 Generating and filtering partitions

`generate_partitions(n, max_part=None)` Recursively enumerates all integer partitions of n.

`subset_partitions(partition)` Returns every partition whose degree does not exceed that of the argument; it simply calls `generate_partitions` for each integer $0 \leq m \leq |\text{partition}|$.

`restrict_partitions(part1_list, part2_list, d_lam)` From two lists keep only pairs (a, b) satisfying $|a| + |b| = d_\lambda$.

`ones_partition(x)` Produces the all-ones partition $(\underbrace{1, \dots, 1}_{x \text{ times}})$; it will later represent the quantities p and q appearing in the socle formula.

All of these functions are purely combinatorial and have no hidden algebraic dependencies.

2.2 2-order Littlewood–Richardson coefficient (`lrcoefP`)

```
def lrcoefP(mu, nu, lam):
    # 1. Skew shape lam / mu
    skew = subtract_partitions(lam, mu)
    if skew is None:          # mu not contained in lambda
        return 0
    # 2. Degree compatibility
    if sum(skew) != sum(nu):
```

```
        return 0
    # 3. Trivial corner cases
    if mu == () and degree(nu) == degree(lam) and nu != lam:
        return 0
    if nu == () and degree(mu) == degree(lam) and mu != lam:
        return 0
    # 4. Enumerate all words of content nu
    entries = weight_to_list(nu)          # e.g. nu =(2,1) ->
       [1,1,2]
    c = 0
    for perm in set(permutations(entries)):
        if is_yamanouchi(perm):
            c += 1
    return c
```

Listing 4: 2-order LR coefficient

The function follows the definition verbatim: * it builds the skew shape, * it checks that the number of boxes matches the size of ν, * it enumerates all distinct permutations of the multiset ν, * and it counts those that satisfy the Yamanouchi condition. Because of the exhaustive permutation step the routine is feasible only for very small partitions (typically $|\nu| \leq 5$).

2.3 4-order Littlewood–Richardson coefficient (`lrcoef4`)

```
def lrcoef4(mu1, mu2, mu3, mu4, lam):
    d_mu1 = degree(mu1); d_mu2 = degree(mu2)
    d_mu3 = degree(mu3); d_mu4 = degree(mu4)
    total1 = d_mu1 + d_mu2          # |a|
    total2 = d_mu3 + d_mu4          # |b|
    parts_a = list(generate_partitions(total1))
    parts_b = list(generate_partitions(total2))
    s = 0
    for a in parts_a:
        for b in parts_b:
            N1 = int(lrcoefP(mu1, mu2, a))    # c^{a}_{mu1,mu2}
            if N1 == 0: continue
            N2 = int(lrcoefP(mu3, mu4, b))    # c^{b}_{mu3,mu4}
            if N2 == 0: continue
            N3 = int(lrcoefP(a, b, lam))      # c^{lambda}_{a,b}
            if N3 == 0: continue
            s += N1 * N2 * N3
    return s
```

Listing 5: 4-order LR coefficient

The double loop over all possible intermediate partitions a and b implements the summation formula displayed earlier. In a production version one would prune the loops using the result of `restrict_partitions` and cache already computed LR numbers; the present script keeps the implementation straightforward.

2.4 Solving the degree balance system (`solveOne`)

The socle filtration imposes linear relations among the degrees of several auxiliary partitions δ, γ, p, q. The system is

$$\begin{aligned} x_1 + 2x_2 + x_3 + x_4 &= k, \\ x_1 + x_2 + x_3 &= k_1 - k_2, \qquad \text{with } x_i \geq 0, \\ x_1 + x_2 + x_4 &= k_3 - k_4, \end{aligned}$$

where

$$x_1 = |\delta|,\ x_2 = |\gamma|,\ x_3 = p,\ x_4 = q,$$

and $k_1 = |\lambda|,\ k_2 = |\lambda'|,\ k_3 = |\mu|,\ k_4 = |\mu'|$.

The function brute-forces the small search space (the indices are bounded by k) and returns every admissible 4-tuple as a dictionary.

2.5 Socle filtration formula (`CalcSoc`)

Let $\lambda, \lambda', \mu, \mu'$ be the partitions supplied by the user and k the filtration index. Define the set $\mathcal{S}(k)$ of all solutions of the above linear system. For a solution (δ, γ, p, q) we form the two 4-order LR coefficients

$$c^{\lambda}_{\lambda', \gamma, \delta, p}, \qquad c^{\mu}_{\mu', \gamma, \delta, q}.$$

The multiplicity of the $(k+1)^{\text{st}}$ socle layer is then

$$\boxed{\sum_{(\delta,\gamma,p,q)\in\mathcal{S}(k)} c^{\lambda}_{\lambda',\gamma,\delta,p}\, c^{\mu}_{\mu',\gamma,\delta,q}}. \tag{1}$$

The implementation follows (1) verbatim:

```python
def CalcSoc(k, lam, lamP, mu, muP):
    k1 = degree(lam);  k2 = degree(lamP)
    k3 = degree(mu);   k4 = degree(muP)

    # Corner case: when a prime partition already has full degree
    if (k1 == k2) or (k3 == k4):
        return 1

    solutions = solveOne(k, k1, k2, k3, k4)
    total = 0
    for sol in solutions:
        d_list = list(generate_partitions(sol["deg delta"]))
        g_list = list(generate_partitions(sol["deg gamma"]))
        p = ones_partition(sol["p"])
        q = ones_partition(sol["q"])
        for d in d_list:
            for g in g_list:
                term1 = lrcoef4(lamP, g, d, p, lam)    # c^{lam}_{
                    lam',gamma,delta,p}
                term2 = lrcoef4(muP, g, d, q, mu)     # c^{mu}_{mu
                    ',gamma,delta,q}
```

```
                    total += term1 * term2
    return total
```

Listing 6: Socle computation

If either $|\lambda| = |\lambda'|$ or $|\mu| = |\mu'|$ the module is already simple, hence the multiplicity equals 1.

2.6 Driver routine (`Master`)

The function `Master` is a thin wrapper that prints a nicely formatted header, invokes `CalcSoc`, and displays the result.

```
def Master(k, lam, lamP, mu, muP, ex_string = ''):
    print(f'''|----------- Calculation for {k+1}-th socle
       filtration ----------|
            lambda = {lam}
            lambda' = {lamP}
            mu = {mu}
            mu' = {muP}
            k = {k}
          ''')
    print(f'>>>>> {k+1}-th Socle Filtration is : {CalcSoc(k, lam,
        lamP, mu, muP)} <<<<<')
    print('-------------------- Done --------------------')
```

Listing 7: Driver

A concrete invocation used for testing is

```
Master(4, (1,1), (), (1,1), ())
```

which computes the 5-th socle layer for $\lambda = \mu = (1, 1)$ and $\lambda' = \mu' = \varnothing$; the output is 4.

---

3 Complexity considerations and practical tips

- **Partition generation.** The number of partitions of n is the partition function $p(n)$, which grows roughly like $\exp\big(\pi\sqrt{2n/3}\big)/(4n\sqrt{3})$. Hence exhaustive generation is only realistic for $n \lesssim 8 - 10$.
- **LR coefficient `lrcoefP`.** The algorithm enumerates all distinct permutations of the multiset described by ν; its complexity is $\dfrac{(\sum \nu_i)!}{\prod \nu_i!}$. For $|\nu| \leq 5$ this is still manageable; beyond that a combinatorial LR implementation (e.g. via the hive model or jeu-de-taquin) is required.
- **Four-order coefficient `lrcoef4`.** Without pruning it performs $|\mathcal{P}(d_{12})| \times |\mathcal{P}(d_{34})|$ calls to `lrcoefP`. A cheap optimisation is to pre-filter the pair (a, b) using `restrict_partitions` before the inner loop.
- **Memoisation.** Many identical triples (μ, ν, λ) appear repeatedly. Storing already computed LR values in a dictionary reduces the overall runtime dramatically.

- **Parallelism.** The outer loops over δ, γ are independent and can be dispatched to multiple processes (e.g. via "multiprocessing.Pool").
- **Typical use-case.** In the original research the program is invoked only for small-degree partitions (often $|\lambda|, |\mu| \leq 4$), where the naive enumeration is perfectly acceptable.

References

1. W. Fulton, *Young Tableaux*, Cambridge Univ. Press, 1997.
2. R. P. Stanley, *Enumerative Combinatorics, Vol. 2*, Cambridge Univ. Press, 1999 (Chapter 7 on LR rule).
3. G. James, A. Kerber, *The Representation Theory of the Symmetric Group*, Addison-Wesley, 1981.
4. J. E. Humphreys, *Representations of Reductive Groups*, Cambridge Univ. Press, 1991 - socle filtration background.